



JAVA SOFTWARE WRAPPER TANUKI

DOCUMENTATION

Table Of Contents

Table of Contents

<i>Project Details</i>	3
<i>Release History</i>	4
<i>Confidentiality Disclaimer</i>	5
<i>Introduction</i>	6
<i>Purpose</i>	6
<i>Tanuki Wrapper Directory Tree</i>	7
<i>Directories Description</i>	8
Bin	8
Conf	8
Drop	8
Lib	9
Logs	9
<i>Pre-Deployment</i>	10
<i>Deployment Process</i>	11
Step 1:	11
Step 2:	12
Step 3:	12
Step 4:	13
Step 5:	13
Step 6:	14
Step 7:	14
Step 8:	15
Step 9:	15
<i>The Application</i>	16
<i>Application Pipeline Settings</i>	16
Step 1:	16
Step 2:	17
Step 3:	18
Step 4:	18
Step 5:	19

Project Details

Application Name	Tanuki Software Wrapper
Project Type	Software Development
Project Version	1.0
Name of Re-leaser	Kennedy N. Gatimu
Prepared By	Kennedy N. Gatimu

Release History

Date	Version	Author	Comments
11/06/2023	1.0	Kennedy Gatimu	Initial Release

Confidentiality Disclaimer

THIS DOCUMENT AND THE INFORMATION IN IT ARE PROVIDED IN CONFIDENCE, FOR THE SOLE PURPOSE OF EQUITY BANK LIMITED AND MAY NOT BE DISCLOSED TO ANY THIRD PARTY OR USED FOR ANY OTHER PURPOSE WITHOUT THE EXPRESS WRITTEN PERMISSION OF EQUITY BANK LIMITED.

Any form of reproduction, dissemination, copying, disclosure, modification, distribution and or publication of this material is strictly prohibited. The contents of this document have not been independently verified and they do not purport to be comprehensive, or to contain all the information that a prospective investor may need. No representation, warranty or undertaking, expressed or implied is or will be made or given and no responsibility or liability is or will be accepted by Equity Bank Limited or by any of its directors, employees or advisors in relation to the accuracy or completeness of this document or any other written or oral information made available in connection with the Company.

Introduction

The Java Service Wrapper is a utility program developed by Tanuki Software, Ltd that is used to launch the JVM in which Java applications runs. You can control various aspects of the application components using settings that control how the Java Service Wrapper runs.

In addition to launching the JVM, the Java Service Wrapper offers features for monitoring the JVM, logging console output, and generating thread dumps.

This document will show the guide for deploying Java applications using Tanuki's Java Service wrapper.

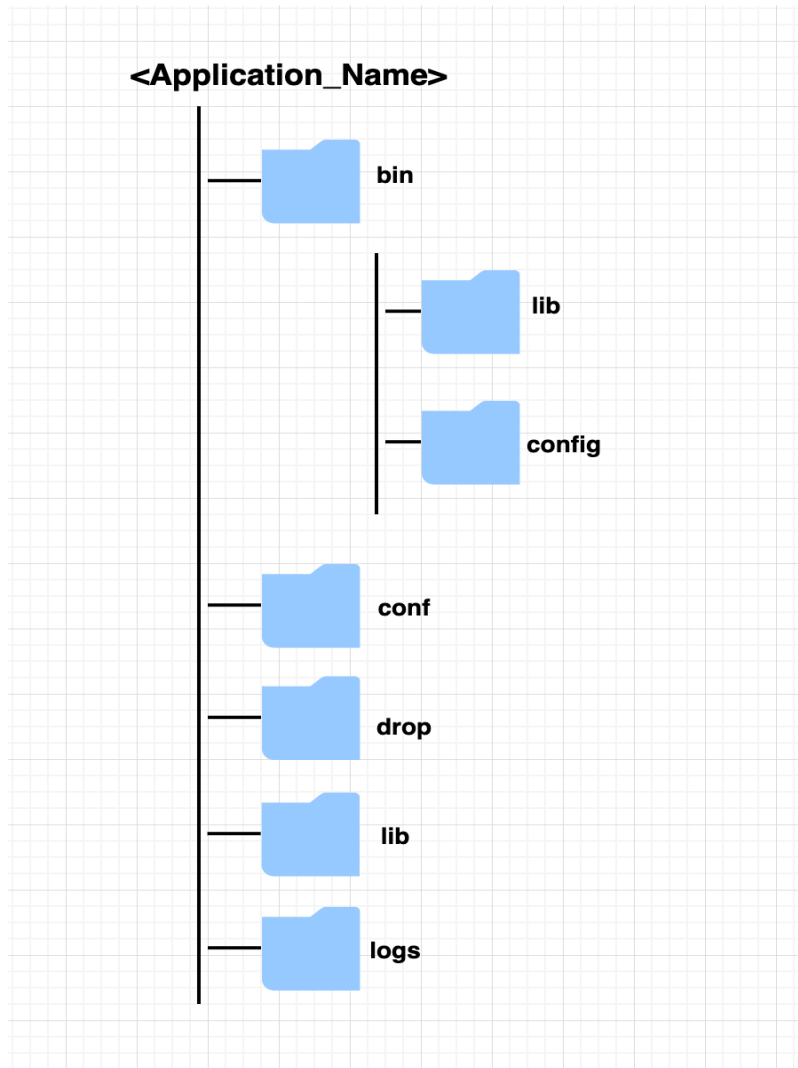
Purpose

The motivation behind idea was due to below, and not limited, challenges:

- i. Tanuki service wrapper service runs as a Unix daemon; hence we are taking advantage of its monitoring and fault-detection capabilities. This ensures that the application being run by Tanuki wrapper is reliable and has a high degree of uptime.
- ii. Application run by Tanuki wrapper are easier to safely start, restart, and stop.
- iii. Using Tanuki wrapper, we can mitigate errors, omissions, and mistakes done during manual deployment and save on time.
- iv. Tanuki wrapper also ensures we don't share application files Insecurely as with manual deployment.

Tanuki Wrapper Directory Tree

Our Java service wrapper, which is based on Tanuki, will create directories as shown in the below tree:



Directories Description

There are five directories which Tanuki wrapper will configure automatically:

- i. Bin
- ii. Conf
- iii. Drop
- iv. Lib
- v. Logs

Bin

This directory will contain:

- i. <application_name>.sh script*
This is the script that will be used to start, stop, or restart the application. Tanuki will automatically create this file and configure the script name to be equal to application name.
- ii. wrapper file*
This is the actual Java Service Wrapper – Tanuki.
It's a binary file which is platform independent.
It is executed first by the Tanuki.
- iii. lib directory*
This directory will hold application specific library jar files.
- iv. config directory*
This should contain the application profile specific properties file.

Conf

This directory holds the configuration file (.conf file).

Tanuki will automatically create this file and configure the conf name to be equal to application name. ie. **<application_name.conf>**

Drop

Drop directory is not usually part of the original Tanuki wrapper software. We have included this directory as a temp directory to temporarily hold the application jar file and lib folder before they are copied to bin folder during application release pipeline run.

Lib

This folder holds Tanuki's specific dependencies:

- i. libwrapper.so (dll)
- ii. wrapper.jar

Logs

This directory holds the log file as configured in the .conf file.
Tanuki will configure this path in the .conf file.

Pre-Deployment

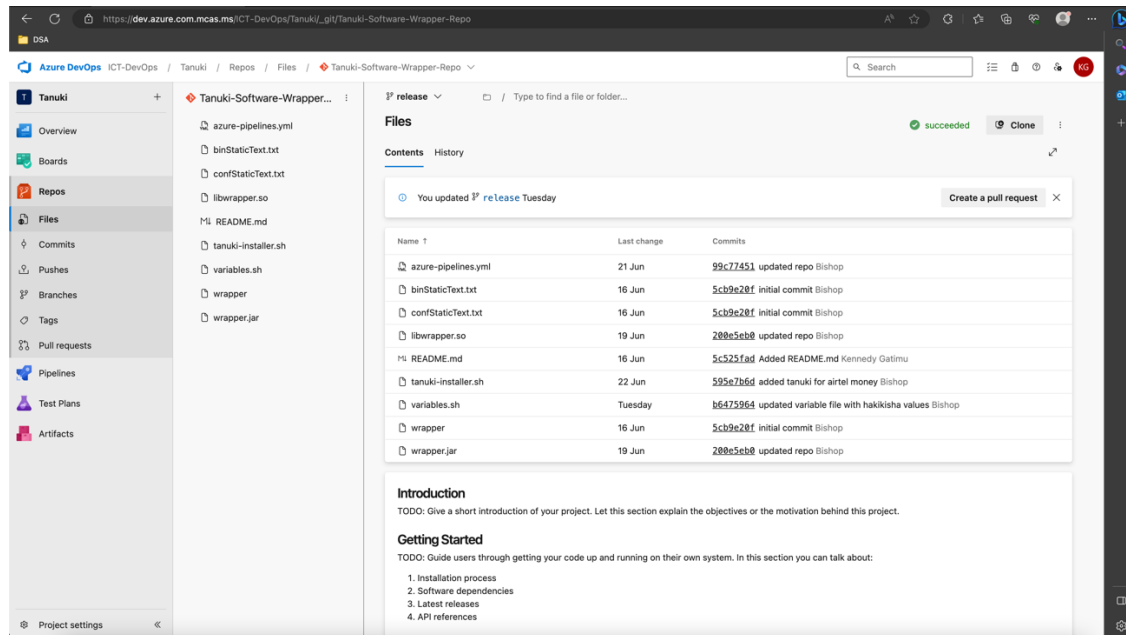
- i. For the First time deployments, always take a backup of the existing application and save it in a separate location.
- ii. Change the ownership of the application home directory to be owned by the deploying user. See the steps in **Step 9** of Deployment process below.

Deployment Process

Step 1:

Login to repo

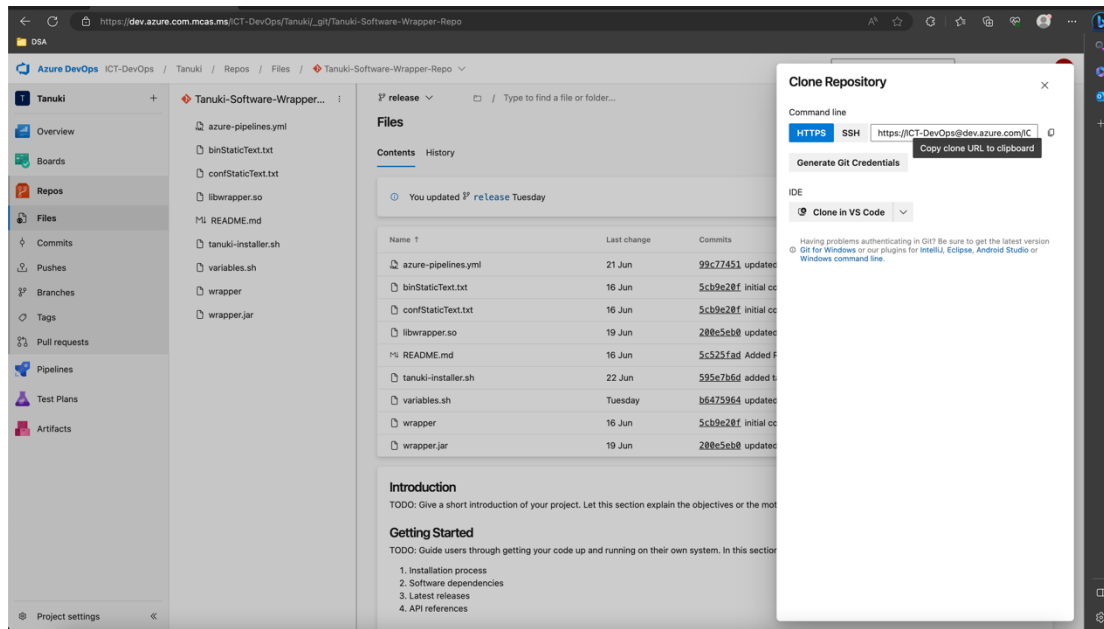
https://dev.azure.com/mcas.ms/ICT-DevOps/Tanuki/_git/Tanuki-Software-Wrapper-Repo



Step 2:

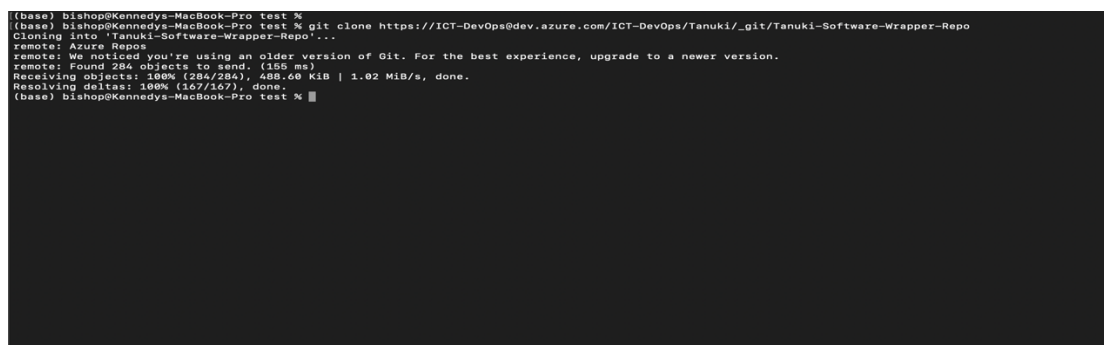
Clone the repo into your local machine.

- i. Click on Clone
- ii. Copy the HTTPS link.



Step 3:

- i. Open terminal and go to the directory where you want to clone the repo to
- ii. Run git clone command:
`git clone https://ICT-DevOps@dev.azure.com/ICT-DevOps/Tanuki/_git/Tanuki-Software-Wrapper-Rep`



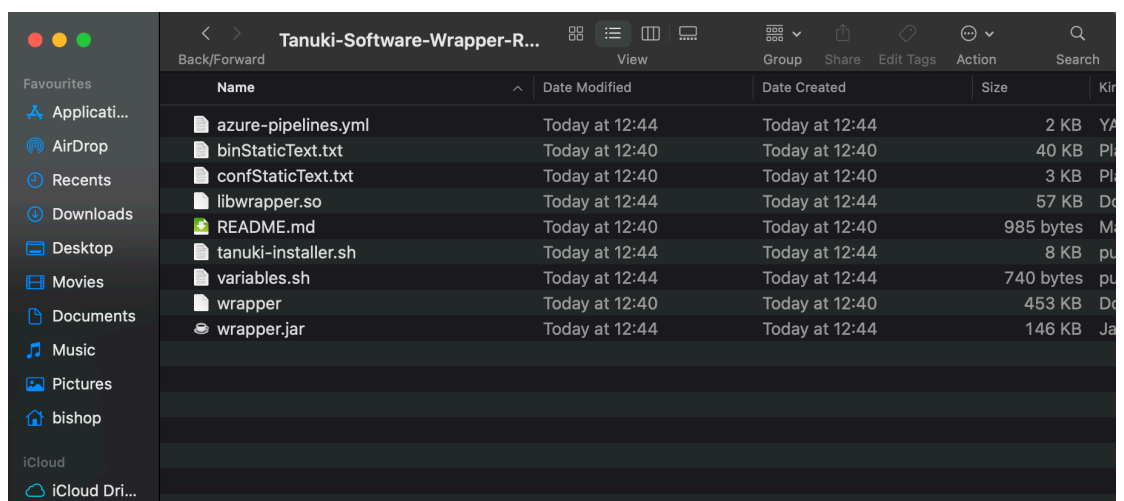
Step 4:

- i. Go to the folder - Tanuki-Software-Wrapper-Repo
`cd Tanuki-Software-Wrapper-Repo`
- ii. checkout to release branch and pull from origin
`git checkout release`
`git pull origin release`

```
base) bishop@Kennedys-MacBook-Pro TEST % cd Tanuki-Software-Wrapper-Repo
base) bishop@Kennedys-MacBook-Pro Tanuki-Software-Wrapper-Repo % git checkout release
branch 'release' set up to track remote branch 'release' from 'origin'.
switched to a new branch 'release'
base) bishop@Kennedys-MacBook-Pro Tanuki-Software-Wrapper-Repo % git pull origin release
from https://dev.azure.com/ICT-DevOps/Tanuki/_git/Tanuki-Software-Wrapper-Repo
* branch          release      -> FETCH_HEAD
already up to date.
base) bishop@Kennedys-MacBook-Pro Tanuki-Software-Wrapper-Repo %
```

Step 5:

- i. Open the directory containing the repo.



Name	Date Modified	Date Created	Size	Kind
azure-pipelines.yml	Today at 12:44	Today at 12:44	2 KB	YAML File
binStaticText.txt	Today at 12:40	Today at 12:40	40 KB	Text File
confStaticText.txt	Today at 12:40	Today at 12:40	3 KB	Text File
libwrapper.so	Today at 12:44	Today at 12:44	57 KB	Dynamic Library
README.md	Today at 12:40	Today at 12:40	985 bytes	Text File
tanuki-installer.sh	Today at 12:44	Today at 12:44	8 KB	Script File
variables.sh	Today at 12:44	Today at 12:44	740 bytes	Script File
wrapper	Today at 12:40	Today at 12:40	453 KB	Folder
wrapper.jar	Today at 12:44	Today at 12:44	146 KB	Jar File

Step 6:

- i. Open **variables.sh** file for editing

NOTE: DO NOT EDIT ANY OTHER FILE!

Step 7:

- i. Edit the variables.sh file with below variable:
 - a. **startClass** – specify the relative path of the main class.
 - b. **javaPath** – Specify the java path in the server (escape forward slash '/' with '\')
- example: Java 8 relative path in .111 uat server is
/u01/gag/jdk/jdk1.8.0_181/bin/java
escaping the forward slash,
javaPath= "Vu01VgagVjdkVjdk1.8.0_181VbinVjava"*
- c. **jarFileName** – Specify the name of the application jar file.
 - d. **applicationName** – Specify the name of the application.
 - e. **applicationPath** – Specify home directory of the application.
 - f. **applicationPathEsc** – Equals to applicationPath above but forward slash is escaped.

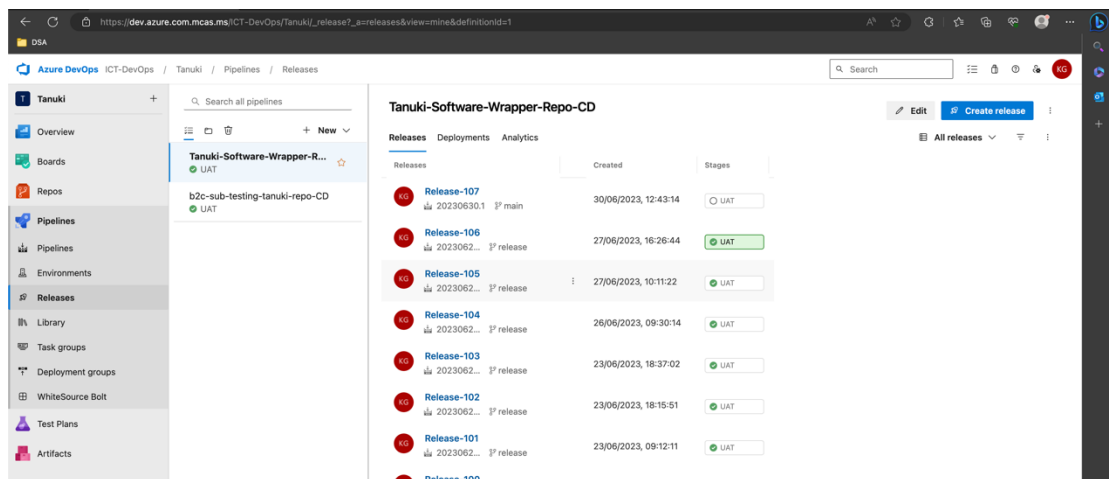
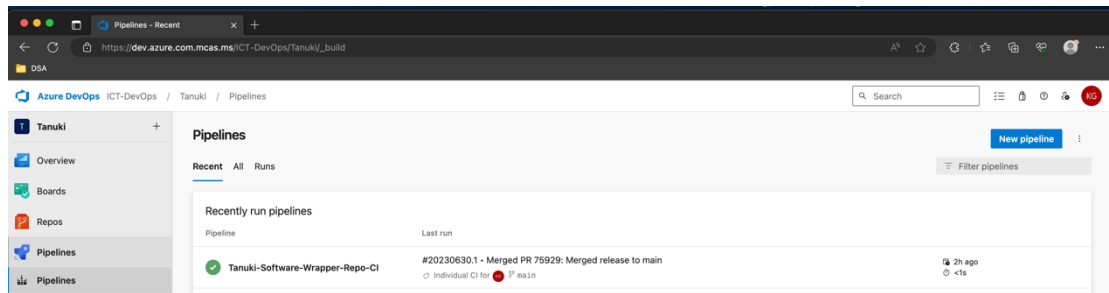
Vairables.sh Example

```
startClass=com.equitybank.pay.mpesa.hakikisha.HakikishaApplication
javaPath="Vu01VgagVjdkVjdk1.8.0_181VbinVjava"
#javaPath="Vu01VgagVjdkVjdk-11.0.11VbinVjava"
jarFileName=hakikisha-0.0.1-SNAPSHOT.jar
applicationName=hakikisha
applicationPath=/u01/gag/
applicationPathEsc="Vu01VgagV"
```

Step 8:

After updating variables.sh, save and do a PR to release branch.

This will run the build pipeline and create and release for the release branch.



Step 9:

- i. Once the pipeline release has been created successfully and application home directory created, change ownership of the application home directory to the user doing the deployment.

In our case, the deploying user is **oracle**.

Example:

ApplicationHomeDirectory= **<applicationPath> + <applicationName>**

```
$ chown -R oracle: oinstall /u01/gag/ hakikisha
```

The Application

After Tanuki has run successfully, we need now to deploy the application in the Tanuki directories.

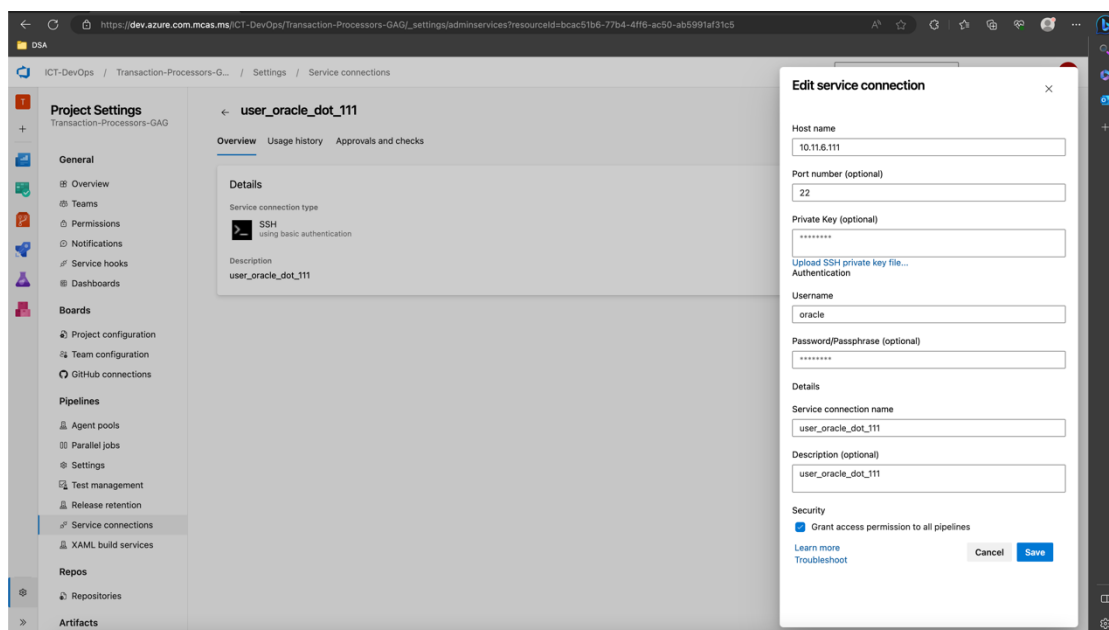
Application Pipeline Settings

For uniformity, when creating application's release pipeline follow the below step:

Step 1:

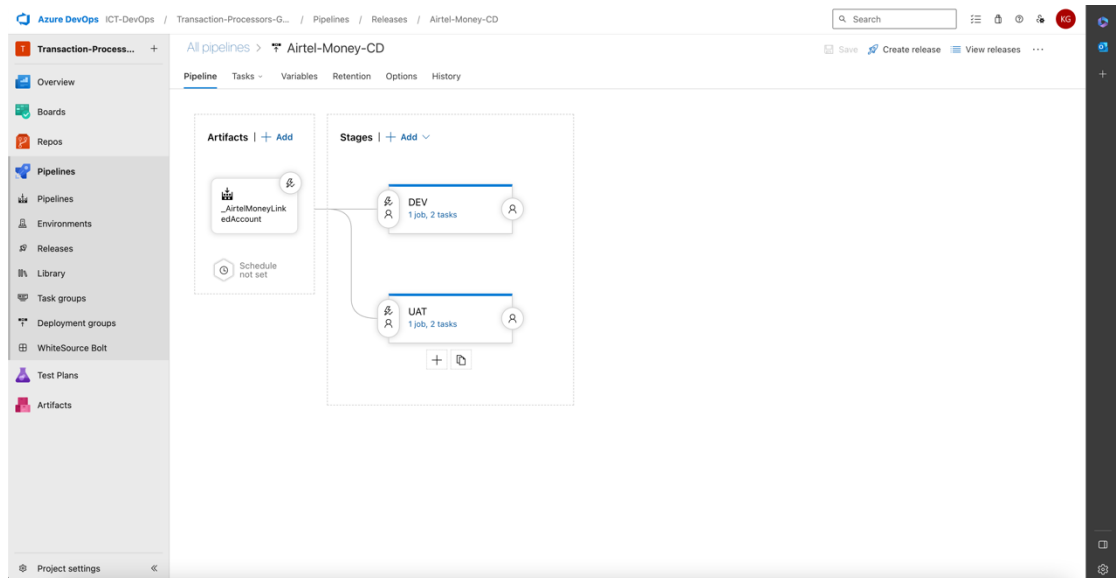
Do this step if ssh connection is not enable, otherwise go to Step 2.

- i. Go to Azure DevOps project that house the desired application repository. In our case, Go to Transaction-Processors-GAG project.
- ii. Go to **Project Settings -> Service connection -> New Service Connection**
- iii. Enter SSh connection Details and Save



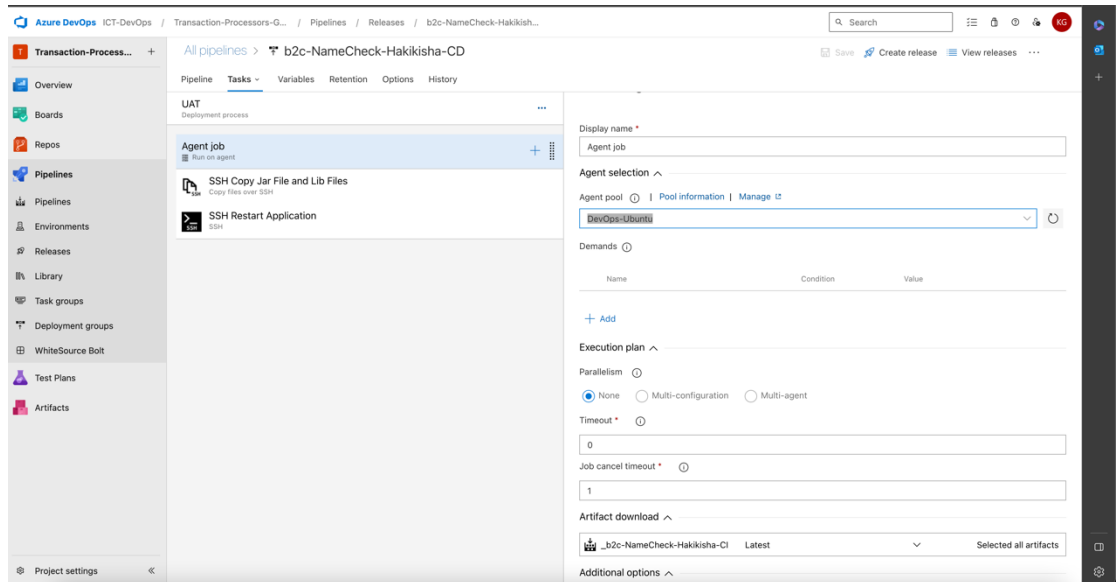
Step 2:

- i. Open the desired repository from Azure DevOps.
- ii. Assuming you have configured the build pipeline, create the release pipeline for the desired branch as shown below:



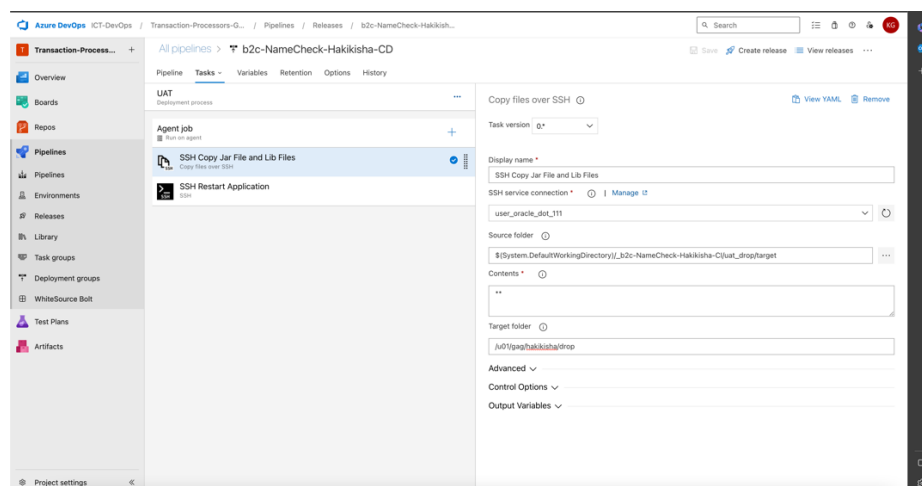
Step 3:

- i. Create Agent Job with Agent Pool = DevOps-Ubuntu



Step 4:

- i. Add a task to the Agent Job – **Copy Files Over SSH**:
Note the Target Folder value: `/u01/gag/<application_name>/drop`
Where:
 - a. **/drop** should always be included as temp directory coinciding with the drop directory in the Tanuki directory
 - b. **application_name= applicationName from variables.sh file**
Example: `/u01/gag/hakikisha/drop`
 - c. **SSH service connection=Service Connection Name** from Step 1 above.



Step 5:

- i. Add a second task to the Agent Job for **SSH Run Shell Command**:
 - a. **SSH service connection=Service Connection Name** from Step 1 above.
 - b. On the Run, select **Inline Script** and paste below commands – inline:

```
cd <applicationPath>/<applicationName>/bin; rm -rf <jarFileName>.jar lib || true;  
cd ../drop; cp -R *.jar lib ../bin/; cd ../bin; ./<applicationName>.sh restart
```

Example:

```
cd /u01/gag/hakikisha/bin; rm -rf hakikisha-0.0.1-SNAPSHOT.jar lib || true; cd ../drop;  
cp -R *.jar lib ../bin/; cd ../bin; ./hakikisha.sh restart
```

Where:

- a. **applicationPath** = applicationPath in the variables.sh file
- b. **applicationName**=applicationName in the variables.sh file
- c. **jarFileName**= jarFileName in the variables.sh file

